

```
dataRostros.py x entrenamiento.py x predicción.py x enviado.py x
1 import cv2
2 import os
3 import mediapipe as mp
4 from tkinter import Tk, Button, Label, messagebox
5 from datetime import datetime
6 import requests
7 import threading
8
9 # URL de la API donde se enviarán los registros
10 API_URL = "http://practiestrella2024.somee.com" # Asegúrate de usar la URL correcta
11
12 # Función para enviar los datos a la API
13 def enviar_registro_api(nombre, tipo):
14     # Usamos 'params' en lugar de 'json' para pasar los parámetros por URL
15     endpoint = "/HoraEntrada" if tipo == "entrada" else "/HoraSalida"
16     params = {"nombre": nombre} # El parámetro 'nombre' se pasa en la URL
17
18     try:
19         response = requests.post(API_URL + endpoint, params=params) # Usamos 'params' para los parámetros en la URL
20         print("Payload enviado:", params)
```

```
if response.status_code == 200:
    messagebox.showinfo(title="Éxito", message=f"Registro de {tipo} enviado correctamente")
else:
    messagebox.showerror(title="Error", message=f"Error {response.status_code}: {response.text}")
except Exception as e:
    messagebox.showerror(title="Error", message=f"No se pudo conectar con la API: {e}")

# Configuración de reconocimiento facial
ruta = 'C:/Users/Evelyn/PycharmProjects/ReconoVisual/Fotos'
etiquetas = os.listdir(ruta)
modelo = cv2.face.LBPHFaceRecognizer_create()
modelo.read("entrenamientoRostros.xml")
detector = mp.solutions.face_detection
dibujo = mp.solutions.drawing_utils

# Función para el reconocimiento de rostros
def reconocer_rostro(tipo):
    cap = cv2.VideoCapture(0)
    if not cap.isOpened():
        messagebox.showerror(title="Error", message="No se pudo acceder a la cámara")
        return
```

```
44 nombre = "Desconocido"
45 with detector.FaceDetection(min_detection_confidence=0.8) as rostros:
46     while True:
47         ret, frame = cap.read()
48         if not ret:
49             break
50
51         frame = cv2.flip(frame, flipCode=1)
52         rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
53         resultado = rostros.process(rgb)
54
55         if resultado.detections:
56             for rostro in resultado.detections:
57                 al, an, _ = rostro.location_data.relative_bounding_box
58                 rect = rostro.location_data.relative_bounding_box
59                 xi = int(rect.xmin * an)
60                 yi = int(rect.ymin * al)
61                 ancho = int(rect.width * an)
62                 alto = int(rect.height * al)
63
64                 cara = cv2.resize(rgb[yi:yi + alto, xi:xi + ancho], dsize=(150, 200))
65                 cara = cv2.cvtColor(cara, cv2.COLOR_BGR2GRAY)
66                 prediccion = modelo.predict(cara)
67
68                 if prediccion[1] < 70:
69                     nombre = etiquetas[prediccion[0]]
70                     color = (0, 255, 0)
71                 else:
72                     color = (0, 0, 255)
73
74                 cv2.rectangle(frame, (xi, yi), (xi + ancho, yi + alto), color, 2)
75                 cv2.putText(frame, nombre, (xi, yi - 10), cv2.FONT_HERSHEY_SIMPLEX, fontScale=0.8, color, thickness=2)
76
77                 cv2.imshow("Reconocimiento Facial", frame)
78                 if cv2.waitKey(1) == 27:
79                     if nombre != "Desconocido":
80                         enviar_registro_api(nombre, tipo)
81                     break
82                 cap.release()
83                 cv2.destroyAllWindows()
84
85 # Función para ejecutar el reconocimiento en un hilo
86 def start_recognition(tipo):
87     thread = threading.Thread(target=reconocer_rostro, args=(tipo,))
88     thread.start()
89
90 # Interfaz gráfica
91 ventana = Tk()
92 ventana.title("Registro de Asistencia")
93 ventana.geometry("400x300")
94 Button(ventana, text="Registrar Entrada", command=lambda: start_recognition("entrada"), bg="green",
95 Button(ventana, text="Registrar Salida", command=lambda: start_recognition("salida"), bg="blue", fg
96 Button(ventana, text="Salir", command=ventana.destroy, bg="red", fg="white").pack(pady=10)
```

```
99 label_resultado = Label(ventana, text="", font=("Arial", 12))
100
101 label_resultado.pack(pady=20)
102
103 ventana.mainloop()
Payload enviado: {'nombre': 'Victor'}
W0000 00:00:1732777025.630553 11568 inferen
Payload enviado: {'nombre': 'Victor'}
```

```
command=lambda: start_recognition("entrada"), bg="green", fg="white").pack(pady=10)
command=lambda: start_recognition("salida"), bg="blue", fg="white").pack(pady=10)
tana.destroy, bg="red", fg="white").pack(pady=10)
Font=("Arial", 12))
```